

Aula 2 — Sistemas Operacionais, Usuários, Grupos e Permissões

1. O que é um Sistema Operacional (visão simples)

Na Aula 1 vimos que a CPU executa instruções e a RAM guarda o que está em uso. Mas alguém precisa **organizar** tudo isso: decidir qual processo roda agora, quem pode acessar qual arquivo, o que cada programa tem permissão de fazer. Esse "alguém" é o **Sistema Operacional (SO)** — Windows, Linux, macOS.

Pense no SO como o **gerente de um prédio**: ele não é o dono de cada apartamento (programa/usuário), mas controla as chaves, decide quem entra em qual sala, e mantém um registro de tudo que acontece (os logs).

Por que importa para segurança: praticamente todo ataque, em algum momento, tenta contornar essa "gerência" — rodar algo sem permissão, acessar um arquivo que não deveria, ou virar "dono do prédio" (elevantar privilégio). Entender como o SO controla acesso é a base para reconhecer quando esse controle foi violado.

2. Usuários

Um **usuário** é uma identidade que o SO reconhece — normalmente ligada a uma pessoa, mas também pode ser uma identidade usada só pelo próprio sistema (chamada de conta de serviço).

- No **Linux**, cada usuário tem um **UID** (User ID) numérico. O usuário `root` tem sempre UID `0` — é o "super usuário", com poder para fazer qualquer coisa no sistema.
- No **Windows**, cada usuário tem um **SID** (Security Identifier), algo como `S-1-5-21-...-1001`. Existem contas especiais do próprio sistema, como `SYSTEM`, que têm privilégios altíssimos, equivalentes (e em certos aspectos superiores) ao `root` do Linux.

Por que importa para segurança: quando você olha um log ou um processo, a primeira pergunta é sempre "**qual usuário fez isso?**". Um processo comum rodando como `root` / `SYSTEM` quando deveria rodar como um usuário comum é um sinal de alerta — pode indicar uma falha de configuração ou uma escalada de privilégio (technique conhecida como **Privilege Escalation**, que vamos estudar em detalhe mais adiante).

3. Grupos

Um **grupo** é uma forma de organizar vários usuários e dar permissões a todos eles de uma vez, em vez de configurar usuário por usuário.

Exemplo no Linux: o grupo `sudo` (Debian/Ubuntu) ou `wheel` (RHEL/Fedora) reúne os usuários que podem executar comandos como `root`. Exemplo no Windows: o grupo `Administrators` reúne os usuários com privilégios administrativos; o grupo `Domain Admins`, em um ambiente de Active Directory, tem controle sobre **todo o domínio** — veremos isso com detalhe no módulo de Active Directory.

Por que importa para segurança: um dos alvos favoritos de um atacante, depois de comprometer uma conta comum, é conseguir entrar em um grupo privilegiado (`Administrators`, `Domain Admins`, `sudo`). Monitorar **mudanças de membros em grupos privilegiados** é uma das detecções mais valiosas em qualquer ambiente — é um exemplo clássico de **Persistence** e **Privilege Escalation** no MITRE ATT&CK, que vamos formalizar no módulo 13.

4. Permissões

Permissão é a resposta à pergunta: "**este usuário pode fazer X neste recurso?**" (ler um arquivo, escrever nele, executá-lo, etc.)

No Linux — modelo clássico (rwx)

Cada arquivo/pasta tem três conjuntos de permissões: dono (**owner**), grupo (**group**) e todos os outros (**others**). Para cada um, três ações possíveis: **read**, **write**, **execute**.

```
-rwxr-xr-- 1 ana  devs 1024 Set 10 10:00 script.sh
```

Lendo da esquerda: dono (`ana`) pode ler/escrever/executar (`rwx`); grupo (`devs`) pode ler e executar (`r-x`); os outros só podem ler (`r--`).

Isso será estudado na prática com o comando `ls -l` e `chmod` no Módulo 01 (Linux para Blue Team) — aqui o objetivo é só entender o conceito.

No Windows — modelo de ACL (Access Control List)

O Windows usa um sistema mais granular chamado **ACL**: uma lista de regras, cada uma dizendo "este usuário/grupo pode fazer isto (ler, escrever, executar, deletar, modificar permissões...) neste arquivo/pasta/chave de Registry". É mais flexível que o modelo rwx do Linux, mas também mais complexo de auditar — o que tem implicação direta para detecção: erros de configuração de ACL são uma causa comum de exposição de dados sensíveis.

5. Princípio do menor privilégio (Least Privilege)

Esse é um dos conceitos mais importantes de toda a cibersegurança:

Cada usuário/processo/serviço deve ter apenas as permissões mínimas necessárias para fazer seu trabalho — nada além disso.

Por quê? Porque se uma conta com privilégio mínimo for comprometida, o dano que o atacante consegue causar é limitado. Se uma conta com privilégio de administrador for comprometida, o atacante já começa com controle quase total.

Ligação com o que vem a seguir: quando estudarmos Threat Hunting e Detection Engineering, uma parte enorme do trabalho é justamente procurar violações desse princípio — contas com privilégio maior do que deveriam ter, ou usadas de forma incomum.

6. Conectando com a Aula 1

```
Sistema Operacional
├─ decide QUEM (usuário/grupo) pode fazer O QUÊ (permissão)
└─ controla os PROCESSOS que vimos na Aula 1
    (cada processo roda "como" um usuário – lembra?)
```

Ou seja: usuário + permissão é o que decide, na prática, o que um processo tem autorização para fazer. Um processo malicioso não ganha poderes mágicos — ele só consegue fazer o que o usuário que o executa tem permissão de fazer. É por isso que "o usuário clicou em um anexo malicioso estando logado como Administrador" é estruturalmente pior do que o mesmo clique feito por um usuário comum.

7. Exercício prático

Se você tiver um terminal Linux/macOS disponível:

```
whoami
id
ls -l /etc/shadow 2>/dev/null || echo "sem permissão (esperado!)"
```

Observe:

1. `whoami` mostra seu usuário atual.
2. `id` mostra seu UID, GID (grupo principal) e todos os grupos aos quais você pertence.
3. A tentativa de listar `/etc/shadow` (onde ficam os hashes de senha no Linux) deve **falhar** para um usuário comum — isso é o princípio do menor privilégio funcionando na prática. Se você conseguiu ler esse arquivo sem ser `root`, algo está mal configurado no sistema.

Guarde o resultado (pode ser um print em `screenshots/00-fundamentos/` se quiser começar a documentar como portfólio) — vamos usar esse mesmo raciocínio quando chegarmos a comandos de enumeração no módulo de Linux.

8. Recapitulando

- SO = quem controla quais usuários/processos podem fazer o quê.
- Usuário = identidade (UID no Linux, SID no Windows); grupo = coleção de usuários com as mesmas permissões.
- Permissão = regra de acesso (rwx no Linux; ACL no Windows).
- Princípio do menor privilégio = dar só o necessário — e é isso que um atacante tenta violar ao escalar privilégio.

Próxima aula

Serviços, arquitetura cliente/servidor, virtualização e containers — como programas ficam "sempre rodando" em segundo plano e por que isso importa tanto para persistência de malware quanto para infraestrutura de defesa (agentes de EDR, coletores de log).

Fontes recomendadas

- **Microsoft Learn** — "**Access Control**": documentação oficial sobre o modelo de ACL, SIDs e privilégios no Windows. <https://learn.microsoft.com/windows/win32/secauthz/access-control>
- **Linux man-pages** — `chmod(1)` e `credentials(7)`: referência oficial do modelo de permissões rwx e do conceito de UID/GID no Linux. <https://man7.org/linux/man-pages/man7/credentials.7.html>

- **NIST SP 800-53, controle AC-6 (Least Privilege)**: definição oficial do princípio do menor privilégio como controle de segurança. <https://csrc.nist.gov/pubs/sp/800/53/r5/upd1/final>